nextwork.org

Build Multi-Region Apps on AWS

Arpita Chowdhury

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Introducing Today's Project!

In this project, I built a production-style multi-region Node.js web application on AWS to demonstrate high availability and disaster recovery principles. I deployed the same application in two AWS regions (us-east-1 and us-west-2) using AWS App Runner with automatic CI/CD deployments triggered from GitHub. The application uses environment variables such as AWS_REGION to dynamically detect which region it is running in, enabling region-aware behavior without hardcoding configuration. This multi-region architecture ensures that if one AWS region experiences an outage, traffic can automatically fail over to the other region, preventing downtime and reducing operational risk. Additionally, serving users from the closest operational region improves latency and overall performance. This project showcases foundational cloud engineering skills in multi-region deployments, automated deployments, resilience design, and cloud-native application configuration.

Key services and concepts

The key tools I used include GitHub for version control and automatic deployments, Node.js and Express.js to build the web application, and AWS App Runner to deploy the app in multiple regions using the AWS Management Console. Key concepts I learned include multi-region architecture for high availability and disaster recovery, automatic CI/CD pipelines triggered by GitHub pushes, how managed services like App Runner handle containers, scaling, and SSL automatically, and how environment variables such as AWS_REGION help applications identify where they are running. I also learned why multi-region deployments improve reliability, reduce latency for global users, and provide fault tolerance during regional outages.

Challenges and wins

This project took me approximately 2 hours. The most challenging part was setting up the GitHub connections and configuring AWS App Runner correctly in multiple regions, especially understanding region-specific resources and troubleshooting deployment errors. It was most rewarding to see the same application running successfully in both us-east-1 and us-west-2 with automatic deployments and region detection working, which clearly demonstrated how multi-region architecture improves reliability, availability, and latency for users.

CH
Arpita Chowdhury
NextWork Student

Creating a Node.js Express App

In this step, I'm going to create a GitHub repository to store my Node.js application code and deploy the application to AWS App Runner in the us-east-1 (Virginia) region. I will configure a GitHub integration with App Runner to enable automatic deployments, so every time I push new code to the repository, the application is automatically rebuilt and deployed without manual intervention. This sets up the foundation for continuous deployment (CI/CD) and ensures the app is publicly accessible through a secure App Runner endpoint, allowing me to verify that the application is running successfully in its first AWS region.

Pushing code to GitHub

I pushed my application code to GitHub by using Git commands to upload my local Node.js project files (including index.js and package.json) to a remote GitHub repository. GitHub stores my code in a centralized version-controlled repository so I can track changes, collaborate with others, and integrate it with AWS App Runner for automatic deployments. This setup enables continuous integration and deployment (CI/CD), where every new code push can trigger an automated build and deployment to AWS, ensuring my application stays updated across multiple regions.

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Deploying to AWS App Runner (us-east-1)

I verified my deployment by opening the default App Runner public URL in my browser and confirming that the page displayed “Hello from AWS!”, which shows that my Node.js app was successfully deployed and running in the us-east-1 region. App Runner automatically provides a public HTTPS endpoint with a managed TLS/SSL certificate, so I didn’t need to configure load balancers, domains, or certificates manually AWS handled the infrastructure, scaling, and security for me.

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Expanding to a Second Region (us-west-2)

In this step, I'm going to deploy the same Node.js app to AWS App Runner in the us-west-2 (Oregon) region so my application runs in multiple geographic locations. Each region needs its own App Runner service and GitHub connection because AWS resources are isolated by region, and services in us-east-1 cannot automatically deploy or manage resources in us-west-2. By running the app in both regions at the same time, I'm creating a multi-region architecture, which improves reliability, reduces latency for users, and ensures the app stays available even if one region experiences an outage.

Creating a region-specific GitHub connection

I created a new GitHub connection named github-connection-west because AWS resources are region-specific, meaning the App Runner service in us-west-2 needs its own secure connection to GitHub to pull code and enable automatic deployments independently from the us-east-1 region.

Benefits of multi-region architecture

I verified both regions by opening the App Runner public URLs for us-east-1 and us-west-2 in my browser and confirming that both endpoints returned "Hello from AWS!", proving that the application is running simultaneously in two separate AWS regions.

CH

Arpita Chowdhury
NextWork Student

Multi-region deployment provides higher availability and fault tolerance, because if one region goes down, the other can continue serving users, and it also improves performance and latency by serving users from the closest geographic region. This architecture is a foundational best practice for building resilient and production-ready cloud applications.

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Adding Region-Aware Responses

In this step, I’m going to update my app to read the AWS region from an environment variable (AWS_REGION) so that I can tell which region is serving each request. After pushing the updated code to GitHub, App Runner will automatically redeploy the application in both us-east-1 and us-west-2, allowing me to verify that each region correctly displays its own region name. This helps demonstrate how multi-region deployments work in real-world systems and how applications can be region-aware for debugging, monitoring, and traffic routing.

Using environment variables for region detection

I used Cursor to update my index.js file so the app reads the AWS_REGION environment variable and dynamically displays which AWS region is responding to the request. I added a fallback value of "local" so the app still works when running on my local machine. The AWS_REGION variable is automatically set by AWS App Runner at runtime, which allows the same codebase to behave differently depending on where it is deployed. Automatic deployment means that every time I push code to GitHub, App Runner automatically rebuilds and redeploys the app in both us-east-1 and us-west-2 without any manual intervention, keeping both regions in sync with the latest code.

Why region visibility matters

I verified region detection by opening both App Runner URLs in separate browser windows and confirming that each deployment returned its respective region name, such as “Hello from us-east-1!” and “Hello from us-west-2!”, which confirmed that the app was correctly reading the AWS_REGION environment variable in each region. This matters for failover because it allows me to confirm which region is actively serving traffic, which is critical during outages or disasters if one region fails, traffic can be routed to the healthy region, ensuring high availability, resilience, and minimal downtime for users.

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Measuring Real-World Latency

In this secret mission, I'm going to measure the round-trip latency from my location to both AWS regions by adding server response timing to my Express app and testing how long each region takes to respond. Latency matters because it directly affects how fast users experience an application lower latency means faster page loads, smoother interactions, and better user experience. Multi-region architecture reduces latency for global users by routing requests to the closest AWS region, minimizing network distance and delays, which is especially important for high-availability, globally distributed applications.

Building a latency test page

I added timing by creating a simple index.html that loads in the browser and pings both regional endpoints (EAST_URL and WEST_URL) when the page opens, then prints the results as plain text. The code measures round-trip latency by recording a start timestamp with performance.now(), sending a lightweight fetch() request to each region (so the browser makes a real network call), and then subtracting the start time from the end time to get the total milliseconds for the request to travel to that region and back. To make those cross-region browser requests work, I also updated index.js to include CORS headers so the app allows cross-origin requests, and I made sure index.html is served from the root path (/) so it shows the latency test page instead of being replaced by an explicit Express route.

Latency results and insights

The latency to us-east-1 was about 204 ms and to us-west-2 was about 74 ms. The closer region had lower latency because network traffic travels a shorter physical distance with fewer internet hops, reducing round-trip time, which is why users experience faster responses when routed to the nearest AWS region.

CH

Arpita Chowdhury
NextWork Student

Project Reflection

I did this project today to learn how to deploy a cloud application across multiple AWS regions, set up automatic deployments from GitHub, and understand how multi-region architecture improves availability, fault tolerance, and latency for users. Another skill I want to learn is traffic routing and failover using services like Amazon Route 53 and AWS Global Accelerator, so I can automatically direct users to the closest healthy region and design production-grade disaster recovery architectures.

The place to learn & showcase your skills

Check out nextwork.org for more projects